FACULTAD DE MATEMÁTICAS - UADY

DESARROLLO DE APLICACIONES WEB

Proyecto Integrador

SLAM SHOES

Prototipo funcional de tienda web para el comercio de calzado deportivo orientado al basquetbol.



Documento de requisitos del software

Primera revisión - 29 de octubre de 2026

Integrantes del equipo:

CAAMAL TENCLE MATEO ALEJANDRO

DZIB MARTÍNEZ AGUSTIN EMANUEL

ESQUIVEL BACELIS ESTEFANIA

MORALES CARO AXEL ADOLFO

Profesor: Juan Pablo Ucán Pech

Repositorio: https://github.com/estefaniaesquivel/Slam-Shoes-Web

PARTE I. DOCUMENTO DE REQUISITOS

1. Nombre de la aplicación

SLAM SHOES. Aplicación web de comercio de calzado deportivo orientado al basquetbol.

2. Descripción general del problema o necesidad

Hoy, muchas personas compran tenis a través de redes sociales, mensajes directos o catálogos sueltos. Esto causa varios problemas:

- No hay un catálogo único y ordenado: hay que buscar modelo por modelo en distintos lugares.
- Es difícil saber qué tallas y colores hay disponibles y cuánto stock queda.
- No existe un carrito donde la persona reúna varios productos y vea el total antes de decidir.
- Los negocios pequeños no tienen una tienda digital propia con identidad visual y datos organizados.

Necesidad que atiende: Slam Shoes ofrece una tienda web con catálogo organizado, cuentas de usuario, carrito de compras y registro de pedidos, accesible desde computadora o celular.

Alcance: es un prototipo escolar. Se busca demostrar la arquitectura completa (FrontEnd, BackEnd y base de datos en la nube) y los flujos principales de compra. Los pagos reales y la logística de envíos quedan fuera del alcance.

Referencia de diseño: la interfaz tomará como inspiración el estilo de tiendas deportivas como Way of Wade, Kicks Own y Anta (inicio de sesión, banners en la página principal de tendencias, navegación clara, menú desplegable, etc). El diseño y los recursos gráficos como el logo serán propios del equipo.

3. Explicación general de la funcionalidad principal

La aplicación tiene dos tipos de usuario: el cliente, que compra, y el administrador, que gestiona los productos. Sus módulos son:

Módulo	**Qué hace**
Inicio (index)	Página principal con menú de navegación, banner, categorías y productos destacados.
Autenticación	Registro e inicio de sesión de usuarios. Las contraseñas se guardan cifradas (hash).
Catálogo	Lista de productos con filtros por categoría, talla, color y precio, y ordenamiento.
Detalle de producto	Fotos, descripción, precio, selección de talla y color, botón de favoritos y agregar al carrito.
Favoritos	Lista de los productos seleccionados como favoritos por el usuario.
Carrito de compras	Agregar, quitar y cambiar cantidades; cálculo automático del total.
Pedidos (checkout simulado para el administrador)	En caso que el usuario sea administrador, hay un módulo más que captura datos de entrega y registro del pedido en la base de datos. El pago es simulado.
Administración de productos	En caso que el usuario sea administrador, hay un módulo más que permite dar de alta, editar o dar de baja productos.

4. Requisitos del software

4.1 Requisitos funcionales

ID	**Requisito**	**Prioridad**
RF-01	El sistema permitirá registrar usuarios con nombre, correo y contraseña.	Alta
RF-02	El sistema permitirá iniciar sesión con correo y contraseña. Las contraseñas se almacenarán cifradas.	Alta
RF-03	El sistema distinguirá dos roles: cliente y administrador.	Media
RF-04	El sistema tendrá una página de inicio con categorías y productos destacados.	Alta
RF-05	El sistema mostrará un catálogo de productos tomados de la base de datos, con filtros y ordenamiento.	Alta
RF-06	El sistema mostrará el detalle de cada producto con sus tallas, colores y existencias.	Alta

RF-07	El usuario podrá agregar productos al carrito, cambiar cantidades, eliminarlos y ver el total.	Alta
RF-08	El usuario autenticado podrá generar un pedido; el sistema guardará el pedido y su detalle.	Media
RF-09	El usuario autenticado podrá consultar su historial de pedidos.	Baja
RF-10	El administrador podrá crear, editar y dar de baja productos.	Baja
RF-11	El BackEnd expondrá una API REST que el FrontEnd consumirá en formato JSON.	Alta

4.2 Requisitos no funcionales

ID	Requisito	Categoría
RNF-01	La interfaz será responsiva (computadora, tableta y celular).	Usabilidad
RNF-02	La navegación entre páginas no recargará el navegador (aplicación de una sola página).	Rendimiento
RNF-03	FrontEnd y BackEnd estarán separados y solo se comunicarán por HTTP/JSON.	Arquitectura
RNF-04	No se guardarán contraseñas en texto plano ni datos de pago reales.	Seguridad
RNF-05	Las credenciales de la base de datos se configurarán con variables de entorno y no se subirán a GitHub.	Seguridad
RNF-06	El código estará versionado en GitHub, con carpetas separadas por capa.	Mantenibilidad
RNF-07	La aplicación podrá desplegarse en servicios gratuitos de la nube.	Despliegue
RNF-08	La aplicación funcionará en las versiones recientes de Chrome, Edge y Firefox.	Compatibilidad

5. Base de datos

La base de datos es relacional, usa PostgreSQL y estará alojada en Supabase. Cuenta con siete tablas relacionadas entre sí por llaves foráneas.

5.1 Tablas y campos

Tabla	Campos principales	Propósito
usuarios	id_usuario (PK), nombre, correo (único), contrasena_hash, rol, fecha_registro	Guarda a clientes y administradores; es la base de la autenticación del sistema.

categorias	id_categoria (PK), nombre, descripcion	Agrupa los productos por tipo (básquetbol, running, casual, etc.).
jugadores	id_jugador (PK), nombre, imagen_url	Almacena a los atletas profesionales que inspiran los modelos de calzado.
productos	id_producto (PK), id_categoria (FK), id_jugador (FK, nulo), nombre, marca, descripcion, precio, imagen_url, activo	Catálogo general de modelos de calzado deportivo.
producto_tallas	id_variante (PK), id_producto (FK), talla, color, stock	Cada combinación específica de talla y color de un producto, con sus existencias disponibles.
pedidos	id_pedido (PK), id_usuario (FK), fecha, total, estado, direccion_envio	Encabezado general de cada compra realizada por los usuarios.
pedido_detalle	id_detalle (PK), id_pedido (FK), id_variante (FK), cantidad, precio_unitario	Renglones de cada pedido: qué producto/variante específica se compró y en qué cantidad.

5.2 Relaciones entre tablas

Relación	Tipo	Explicación
categorias -> productos	1 a N	Una categoría tiene muchos productos; cada producto pertenece a una categoría.
jugadores -> productos	1 a N	Un jugador profesional inspira o está asociado a muchos modelos de calzado; cada modelo puede pertenecer a un jugador específico (o ser nulo).
productos -> producto_tallas	1 a N	Un producto cuenta con muchas variantes de talla, color y stock.
usuarios -> pedidos	1 a N	Un usuario registrado puede realizar muchos pedidos a lo largo del tiempo.
pedidos -> pedido_detalle	1 a N	Un pedido contiene uno o más renglones o líneas de compra.
producto_tallas -> pedido_detalle	1 a N	Una variante específica de talla y color puede aparecer en los renglones de pedidos de distintos clientes.

5.3 Diagrama entidad-relación

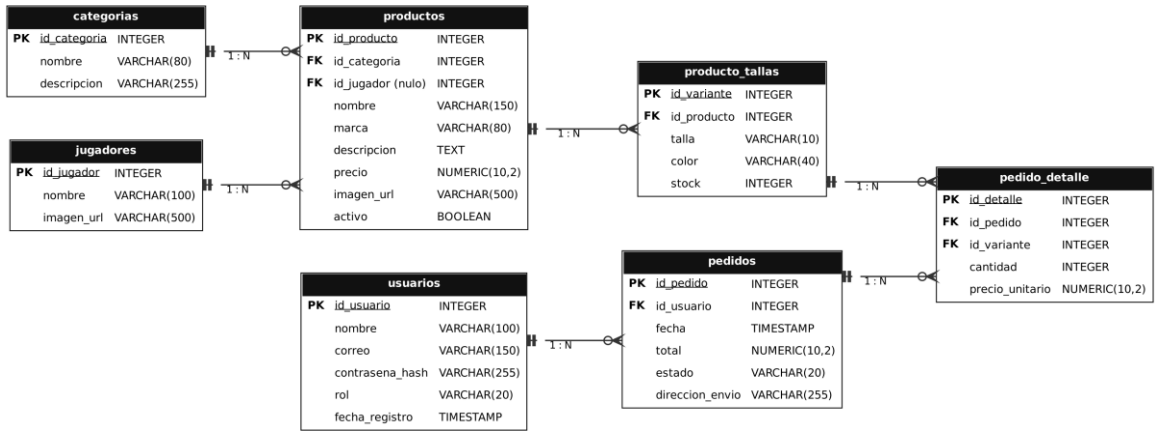


Figura 1. Diagrama entidad-relación de Slam Shoes.

El documento del modelo relacional (tipos de dato, llaves y restricciones de cada tabla) se entregará por separado, como lo pide el proyecto). La explicación del diagrama entidad-relación y sus notaciones se encuentra en tal documento: SLAM SHOES MODELO RELACIONAL

6. Integración futura del FrontEnd y el BackEnd

El FrontEnd (Vue) y el BackEnd (Spring Boot) serán dos aplicaciones independientes que se comunican por una API REST: Vue envía peticiones HTTP y Spring Boot responde con datos en formato JSON. El BackEnd es el único que habla con la base de datos.

6.1 Endpoints previstos de la API

Método	**Ruta**	**Descripción**
POST	/api/auth/registro	Registra un usuario nuevo.
POST	/api/auth/login	Valida correo y contraseña y devuelve los datos del usuario y su rol.
GET	/api/productos	Lista los productos (con filtros opcionales).
GET	/api/productos/{id}	Detalle de un producto con sus tallas y stock.

POST	/api/pedidos	Registra un pedido a partir del carrito.
GET	/api/pedidos/usuario/{id}	Historial de pedidos de un usuario.
POST / PUT / DELETE	/api/productos	Alta, edición y baja de productos (administrador).

6.2 Pasos de integración

1. Crear el proyecto en Supabase.
2. Configurar en el BackEnd la conexión a Supabase con variables y hacer pruebas mediante Bruno API Client para verificar la funcionalidad de la BD.
3. Crear en Spring Boot los controladores, servicios y repositorios de usuarios y productos.
4. Crear en Vue las vistas (inicio, login, registro, catálogo, detalle, carrito) y las rutas.
5. Crear el store del carrito y el de sesión de usuario con Pinia.
6. Mediante la configuración de CORS, conectar las vistas con la API y probar todo en local (FrontEnd en localhost:5173, BackEnd en localhost:8080).
7. Desplegar el BackEnd y Frontend en Render, y actualizar la configuración de CORS.

7. Estado actual y primer avance de codificación

Como parte de la fase inicial de desarrollo del prototipo funcional de Slam Shoes, se ha concretado la implementación de la estructura base del FrontEnd utilizando HTML, Bootstrap para el diseño responsivo y la maquetación de componentes, y JavaScript para la lógica de interacción del cliente, incluyendo un sistema de carrito de compras con persistencia local (que se adaptará en la siguiente entrega a una persistencia a base de datos).

A continuación, se detallan los componentes principales desarrollados en este primer avance:

7.1. Página Principal (index.html)

- Barra de navegación (navbar): Integrada con un menú colapsable para dispositivos móviles, menús desplegables (dropdowns) clasificados por marcas y jugadores profesionales, barra de búsqueda, enlace directo al módulo de autenticación y un indicador dinámico de cantidad de artículos en el carrito.

- Sección multimedia y carruseles: Estructuración de carruseles dinámicos mediante componentes de Bootstrap para mostrar novedades, ofertas destacadas y colecciones de calzado de básquetbol.

- Catálogo interactivo en tendencia: Visualización de tarjetas de productos con soporte para el botón "Añadir al carrito".

- Panel lateral del carrito: Un cajón deslizante lateral que muestra los artículos seleccionados, subtotales, cálculo del monto total en pesos mexicanos (MXN), opción de vaciar el carrito y un botón de redirección para continuar con el flujo de compra.

7.2. Módulo de Autenticación (login.html)

- Interfaz de acceso: Formulario limpio y centrado para el inicio de sesión de usuarios, solicitando correo electrónico y contraseña.

- Simulación de sesión: Incorporación de un script en JavaScript que intercepta el evento submit del formulario, captura el correo ingresado y emite una alerta de confirmación antes de redirigir al usuario de vuelta a la página principal (index.html), simulando el flujo de entrada al sistema.

7.3. Lógica de negocio del cliente (carrito.js)

- Persistencia local: Utilización de localStorage bajo la clave slam_shoes_cart para garantizar que los productos agregados al carrito no se pierdan si el usuario recarga la página o navega entre secciones.

Funciones principales implementadas:

- obtenerCarrito() y guardarCarrito(): Lectura y escritura segura del estado actual del carrito en formato JSON.
- actualizarBadge(): Modifica dinámicamente el contador numérico en el icono de la bolsa de compras.
- renderizarCarrito(): Genera de forma dinámica los elementos HTML dentro del panel lateral, calculando subtotales y sumatorias totales.
- agregarAlCarrito() y eliminarItem(): Controlan la adición de nuevas unidades (incrementando la cantidad si el producto ya existía) y la supresión de artículos individuales.